SI-AT-006 — Toward Policy-Driven Structural Hybrid Intelligence

Series: Structural Intelligence — Additional Thoughts
Release Track: Post-Release Structural Delta
Status: Synthesis Direction and Candidate Interpretation of Point 12
Version Context: SI Compass v0.2.0

Sizhe Tan
with chatGPT (OpenAI)
2026-08-20

Repository: https://github.com/sizhet/Structural-Intelligence-Compass

Abstract

The Structural Intelligence Compass identifies multiple computational structures, operators, and research directions that were initially developed as largely distinct discoveries.
As these structures mature, a new pattern becomes visible.
They begin to converge.
Differential organization, Calling-Path Folding, Enumerative Structure, Grounded Structural IR, Local Learned Intelligence, Two-Way CCC, Calling Graphs, runtime switching, and Per-Node Intelligence are no longer merely independent mechanisms.
They increasingly meet at shared computational junctions.
At those junctions, a new problem appears:
How should heterogeneous forms of intelligence be structurally organized, selectively invoked, combined, validated, and evolved?
This article proposes **Policy-Driven Structural Hybrid Intelligence** as a candidate interpretation of the still-open synthesis direction represented by Point 12 of the Structural Intelligence Compass.
The proposed architecture is not Hybrid AI in the weak sense of placing several technologies in the same system.
It is a policy-controlled computational organization in which:

- physical observations become Grounded Structural IR;
- possibility spaces are enumerated;

- structures are organized and localized;
- multiple intelligence mechanisms coexist at nodes;
- policy governs selection, generation, composition, validation, and retirement;
- runtime determines actual execution;
- measurement and feedback drive local and global adaptation.

The central transition is:
Structural Discovery → Structural Synthesis → Structural Evolution
The hypothesis is that the next frontier of Structural Intelligence may not be another isolated intelligence mechanism, but the organized runtime ecology formed when many such mechanisms are allowed to cooperate under policy.

1. Why Point 12 Should Remain a Synthesis Point

An open research point has value when its final form has not yet been forced prematurely.
Point 12 should therefore not be treated as:
Missing Topic
  |
  v
Insert Technology X
A more natural interpretation has gradually emerged:
Points 1–11
  |
  v
Multiple Structural Discoveries
  |
  v
Convergence
  |
  v
Point 12
  |
  v
Synthesis
Under this interpretation, Point 12 is not merely another technology.

It asks:
What happens when the technologies and structural principles identified elsewhere in the Compass begin to operate together?
This makes Point 12 qualitatively different from an isolated technical item.
It becomes a **Synthesis Point**.

2. From Structural Discovery to Structural Synthesis

Much of the Structural Intelligence research program has followed a discovery-oriented pattern:

Observe Computational Phenomenon
|
v
Identify Structure
|
v
Isolate Principle
|
v
Define Operator
|
v
Generalize

This produced structures and concepts such as:

- structural recognition;
- Differential Trees;
- Common Concept Core;
- Two-Way CCC;
- Function Tunnels;
- runtime invariants;
- Calling Graphs;
- selective runtime reachability;
- policy-driven organization;

- localized intelligence.

The research problem now changes.
Once many useful structures exist, the question becomes:
How should they be assembled?
This requires a new methodology:
Existing Structures
|
v
Identify Junctions
|
v
Define Interfaces
|
v
Expose Strategy Space
|
v
Apply Policy
|
v
Measure Runtime Behavior
|
v
Optimize Composition
This is **Structural Synthesis**.

3. Hybrid Intelligence Is Not Component Accumulation

A weak definition of Hybrid AI is:
LLM
+
World Model
+
Search
+
Symbolic System
This describes coexistence.
It does not describe organization.
A structurally meaningful hybrid system must answer:

Why are these components connected?

Where do they meet?

Who controls their interaction?

Which component has runtime authority?

How are conflicts resolved?

How is performance measured?

How does the architecture change over time?
Therefore:
Hybridization without organization is only component accumulation.
Structural Hybrid Intelligence requires explicit computational relationships.

4. The Emerging Structural Stack

Recent SI extensions suggest the following broad computational stack:

Physical World
    |
    v
Grounding
    |
    v
Grounded Structural IR
    |
    v
Enumerative Possibility Space
    |
    v
Structural Organization
    |
    v
Calling / Routing
    |
    v
Per-Node Intelligence Space
    |
    v

Policy
|
v
Runtime Reachability
|
v
Execution
|
v
Measurement
|
v
Feedback
Each layer performs a distinct computational job.
This layered view helps prevent unnecessary paradigm conflict.

5. Grounding: Bringing the World Into Computation

Physical systems begin with observations:
Vision
Sound
Radar
Lidar
Motion
Force
Biological Signals
These must become usable structure.
Thus:
Physical World
|
v
Sensors
|
v
Learned Perception
|
v
Grounded Structural IR
This gives Structural Intelligence access to the physical world.
Without grounding, higher-level structural computation lacks material on which to operate.

6. Enumeration: Creating the Possibility Space

Once a usable representation exists, a system may need to construct alternatives:

Possible States

Possible Paths

Possible Candidates

Possible Orders

Possible Actions

This produces:

Structural IR

|

v

Enumeration

|

v

Possibility Space

Enumeration is therefore a major upstream structural layer.

It creates the raw alternatives over which intelligence can operate.

7. Organization: Making Possibility Manageable

A large possibility space is often computationally unusable in flat form.

Structural organization introduces:

Hierarchy

Difference

Equivalence

Buckets

Locality

Pruning

Examples include:

- Differential Trees;

- BTP;
- classification structures;
- state aggregation.

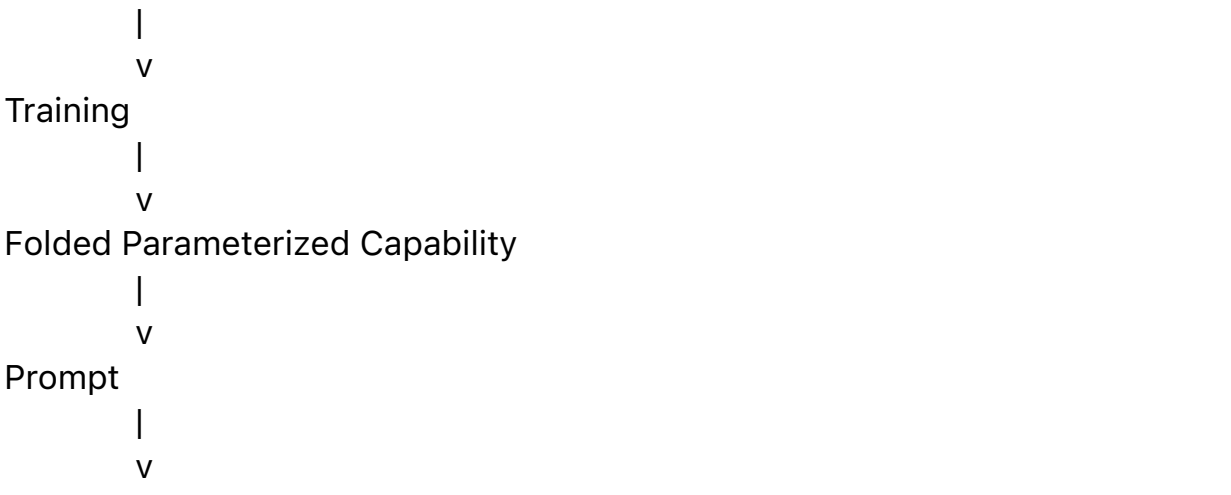
Thus:
Possibility Space
|
v
Organization
|
v
Localized Structure
This is where complexity begins to become manageable.

8. Calling: Exposing Computational Reachability

Once structures are organized, the system must determine what can call what.
Calling Graphs and Calling Paths provide:
Potential Reachability
|
v
Executable Paths
A Calling Graph may encode:
A -> B
A -> C
B -> D
C -> E
while runtime may expose only one path:
A -> C -> E
Thus Calling Structure connects static organization with runtime behavior.

9. Folding: Concentrating Capability

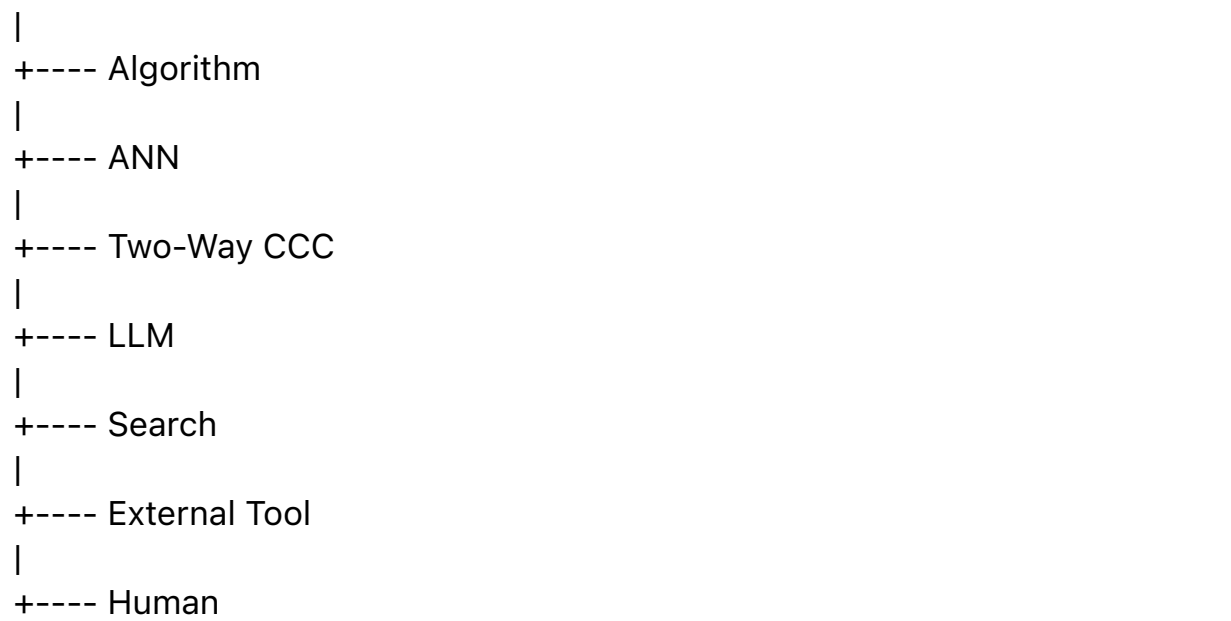
Large Language Models introduce another major structural mechanism.
They provide:
large-scale Calling-Path Folding over language-mediated intelligence.
A simplified view is:
Massive Language / Knowledge Space



This creates broad reusable intelligence.
But folded intelligence must still be placed correctly inside larger systems.

10. Local Intelligence: Bringing Capability to the Node

A localized node may possess multiple intelligence mechanisms:



This is not a single model.
It is an **Intelligence Space**.
The node may also generate new intelligence from local experience.
Thus:
Per-Node Intelligence is broader than Per-Node Model.

11. Learn-on-Demand as a General Capability

Local intelligence may emerge when needed.

For ANN:

Local Data
  |
  v
Train
  |
  v
Local Model

For Two-Way CCC:

Positive / Negative Experience
      |
      v
Preserve / Subtract
      |
      v
Local Structural Invariant
      |
      v
Two-Way CCC

For policy:

X-Y-M History
  |
  v
Policy Learning

Thus:

Learn-on-Demand is not a neural-network-specific idea.

It is a general capability of a localized computational organization.

12. The Intelligence Space

The Intelligence Space is a central object in Structural Hybrid Intelligence.

A node may contain:

Intelligence Space
 |
 +---- Exact Algorithm
 |
 +---- Local ANN
 |
 +---- Statistical Model
 |
 +---- CCC

|
+---- Two-Way CCC
|
+---- LLM
|
+---- Search
|
+---- Tool
|
+---- Human

These mechanisms differ in:

- accuracy;
- latency;
- cost;
- scope;
- explainability;
- adaptability;
- provenance;
- robustness.

There is therefore rarely one globally dominant solver.

13. The Policy Surface

Once several mechanisms become viable, a Policy Surface appears.
Policy must answer:
Which solver?

When?

Where?

In what order?

In parallel or sequentially?

With what budget?

With what validation?

With what fallback?
Thus:
Intelligence Space
    |
    v
Policy Surface
    |
    v
Runtime Strategy
This is the operational center of the emerging hybrid architecture.

14. Policy Does More Than Select

Policy should not be reduced to:
Choose A or B
A mature policy may:
Select

Generate

Combine

Sequence

Parallelize

Validate

Fallback

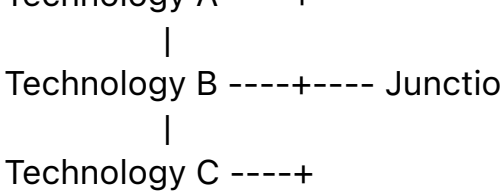
Escalate

Retire
Thus:
Policy

|
+---- Select Intelligence
|
+---- Generate Intelligence
|
+---- Compose Intelligence
|
+---- Validate Intelligence
|
+---- Retire Intelligence
This makes Policy a general control mechanism over computational capability.

15. The Computational Junction

The key research object may therefore be the **Junction**.
Consider:



The junction creates:

- alternatives;
- composition choices;
- ordering choices;
- validation choices;
- resource tradeoffs.

Therefore:
The basic research object of Structural Hybrid Intelligence may not be the component, but the junction.
This represents a major shift from component-centric research.

16. Junctions Create Strategy Spaces

If three mechanisms are available:
A

B
C
possible strategies may include:
A

B

C

A -> B

B -> C

A || C

A -> B -> C
Therefore:
Convergence
  |
  v
Strategy Enumeration
  |
  v
Policy Surface
This links Structural Hybrid Intelligence directly to Enumerative Structure.

17. Differential Trees and LLM Folding as a Foundational Convergence

One of the most important convergences occurs between:
Differential Organization
and:
LLM Calling-Path Folding
Their roles are complementary.
Differential Trees:
localize structure.
LLMs:
provide broad folded intelligence.
Thus:
Input
 |

v
Differential Routing
|
v
Localized Context
|
v
LLM
|
v
Result
This architecture helps explain the emergence of Brain Unit AI.

18. Brain Unit AI as Structural Hybrid Intelligence

A mature Brain Unit may contain:

Brain Unit
|
+---- Problem Scope
|
+---- Local Memory
|
+---- Structural Organization
|
+---- Intelligence Space
|
+---- Policy
|
+---- Runtime
|
+---- Measure
|
+---- Feedback

This is not simply a small model.
It is:
a localized organizational runtime with access to multiple forms of intelligence.
That distinction is essential.

19. X-Y-M as Runtime Memory

A node may record:

X = local context

Y = selected computational strategy

M = measured result
Repeated runtime experience creates:
(X1, ANN, M1)

(X2, CCC, M2)

(X3, LLM, M3)

(X4, ANN -> CCC, M4)
This enables a new form of learning:
not merely learning the answer, but learning which computational strategy tends to work best.

20. Level-1 and Level-2 Intelligence

This distinction can be formalized.

Level-1 Intelligence

Solve the problem.
For example:
f(X) -> Y

Level-2 Intelligence

Decide how the problem should be solved.
For example:
P(X, Node, State, Constraints)
|
v
Computational Strategy
|
v
Y
Structural Hybrid Intelligence is strongly concerned with Level-2 Intelligence.

21. From Best Model to Best Strategy

Traditional model evaluation often asks:
Which model has the highest score?
A hybrid runtime asks:
Which strategy is best under current constraints?

Possible constraints include:
Accuracy
Cost
Latency
Energy
Risk
Privacy
Explainability
Novelty
Validation
Therefore:
There may be no globally best intelligence mechanism. There may only be a best computational strategy under a particular policy.

22. Policy Optimization

Policy can therefore be treated as an optimization problem.
Conceptually:
P* = best policy under objectives and constraints
The objective may balance:
Utility
=
Benefit
- Cost
- Latency
- Risk
- Validation Burden

Different systems will produce different optima.
Thus Policy-Driven Hybrid Intelligence is inherently application-sensitive.

23. Policy Can Begin Simple

A policy need not begin as reinforcement learning.
A sensible evolution may be:
Rules
|
v
Thresholds
|
v
Statistics
|

v
Contextual Routing
|
v
Learned Policy
|
v
Adaptive Runtime Policy
This supports a Minimal Evolution Threshold approach:
Use the simplest policy mechanism that adequately controls the junction.

24. Policy as the Control Plane

A useful systems distinction is:
Data Plane
->
Actual computation
versus:
Control Plane
->
Policy, routing, authority, validation
Under this interpretation:
Policy-Driven Organization provides the control plane of Structural Hybrid Intelligence.
This gives PDOS-style organizational research a direct role in the emerging Point 12 architecture.

25. Runtime Authority

Policy proposes a strategy.
Runtime determines what actually happens.
Thus:
Potential Intelligence Space
|
v
Policy
|
v
Runtime Reachability
|
v

Execution
This connects Policy-Driven Structural Hybrid Intelligence with:

- Runtime Computational Primitives;
- selective reachability;
- triggering;
- switching;
- FTRI.

The hybrid architecture is therefore not static.
It is runtime-controlled.

26. Validation Is Part of the Strategy

A policy may choose not only a solver but also a validation chain.
For example:

```
Low-Risk Input
    |
    v
Local ANN
```

versus:

```
High-Risk Input
    |
    v
Local ANN
    |
    v
Two-Way CCC
    |
    v
LLM / Human Review
```

Thus:
Validation itself belongs inside the strategy space.
This is especially important in high-consequence systems.

27. Human Intelligence as a Structural Resource

Human review can be treated as one possible runtime component:
Machine Confidence High

|
v
Automatic Execution
Machine Confidence Low
|
v
Human Escalation
This avoids placing humans outside the computational architecture.
Instead, they become part of the structured intelligence space.

28. Structural Convergence and Runtime Convergence

Two different forms of convergence should be distinguished.

Structural Convergence

Two technologies combine into a new architecture.
Example:
Differential Tree
+
LLM
|
v
Localized Brain Unit

Runtime Convergence

Several tools compete or cooperate at one runtime junction.
Example:
ANN

CCC

LLM

Algorithm
Thus:
Structural Convergence
->
New Structure
while:
Runtime Convergence
->
Policy Surface
Both contribute to Structural Synthesis.

29. Grounding, World Models, and Runtime Synthesis

Grounded intelligence adds another major junction.

A physical system may contain:

```
Sensors
  |
  v
Grounded Structural IR
  |
  v
World Model
  |
  v
Possible Futures
  |
  v
Search / Planning
  |
  v
Policy
  |
  v
Runtime Action
```

Different computational forms naturally divide labor:

```
Learned Perception
   ->
Grounding

World Model
   ->
State Dynamics

Enumeration
   ->
Possible Futures

Search
   ->
Exploration
```

Policy
  ->
Selection

Runtime Control
  ->
Execution
This is Hybrid Intelligence through structural placement rather than paradigm competition.

30. The End of Unnecessary Paradigm Wars

Many AI debates are framed as:
LLM vs World Model

Neural vs Symbolic

Learning vs Search

General Model vs Specialized Model
Structural Intelligence suggests that these often compare different computational jobs.
A more useful question is:
Where does each technology belong in the computational landscape?
For example:
ANN
  ->
Local Learned Mapping

LLM
  ->
Folded General Intelligence

CCC
  ->
Explicit Structural Recognition

World Model
  ->
Grounded Dynamics

Search
  ->
Possibility Exploration

Calling Graph
  ->
Reachability

Policy
  ->
Strategy Control

FTRI
  ->
Runtime Switching
This converts competition into composition.

31. Structural Placement as a Core Principle

A useful principle follows:
Many AI paradigm conflicts are not conflicts of capability, but failures of structural placement.
A technology can be excellent and still be misplaced.
Structural Hybrid Intelligence therefore asks:
What job?

At what layer?

At which node?

Under which policy?

With which validation?
This is a more operational question than:
Which paradigm is true AI?

32. From Model Architecture to Intelligence Operating Architecture

Once a system contains:

- grounding;

- structural representation;
- enumeration;
- organization;
- multiple solvers;
- policy;
- runtime;
- validation;
- feedback;

it begins to resemble something larger than a model architecture.
A useful working term is:
Intelligence Operating Architecture
Conceptually:
World / Input
    |
    v
Structure
    |
    v
Possibility Space
    |
    v
Organization
    |
    v
Intelligence Space
    |
    v
Policy Control
    |
    v
Runtime
    |
    v

Feedback
This is an architecture for operating heterogeneous intelligence.

33. From Brain Unit to Computational Organization

A Brain Unit can therefore be reinterpreted as a small organization.
It owns:
A Local Problem Region

Local Memory

Local Intelligence

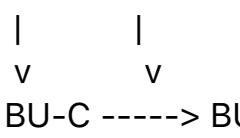
Local Policy

Local Runtime

Local Measurement
Thus:
A Brain Unit is not merely a small AI. It is a small intelligent organization.
This distinction becomes increasingly important as systems scale.

34. Multiple Brain Units Form a Runtime Network

Many Brain Units may be connected:



Calling Graphs describe potential reachability.
FTRI-style mechanisms may control runtime switching.
Policy may alter paths.
Thus the global system becomes:
Localized Intelligence
+
Networked Organization
+
Runtime Policy
This is structurally different from one monolithic model.

35. From Runtime Network to Intelligence Ecology

If nodes can:

- specialize;
- generate local intelligence;
- change policy;
- select different tools;
- retire poor solutions;
- create new paths;

then the system begins to behave like an ecology.
A computational ecology contains:
Multiple Local Intelligences

Multiple Policies

Multiple Runtime Paths

Competition

Cooperation

Adaptation

Replacement
This suggests:
Policy-Driven Structural Hybrid Intelligence may eventually evolve into a Computational Intelligence Ecology.

36. Structural Evolution

The progression does not stop at synthesis.
Suppose a policy repeatedly discovers:
A -> B -> C
as a successful strategy.
That path may eventually become:
New Unit F
through:

- compilation;

- caching;
- learned model formation;
- CCC formation;
- workflow packaging;
- API creation.

Thus:
Runtime Experience
|
v
Repeated Successful Structure
|
v
Folding
|
v
New Computational Unit
The system has evolved structurally.

37. Discovery → Synthesis → Evolution

This produces a larger research progression:
Structural Discovery
|
v
Find useful computational structures
|
v
Structural Synthesis
|
v
Organize and combine structures
|
v
Structural Evolution
|

v
Allow successful runtime patterns to become new structures
This may represent the long-term developmental path of Structural Intelligence itself.

38. Point 12 as a Candidate Synthesis Layer

Point 12 can now be interpreted provisionally as:
The policy-driven synthesis of heterogeneous computational intelligence.
Its objects are not limited to models.
They include:
Computational Junctions

Intelligence Spaces

Policy Surfaces

Runtime Compositions

Validation Paths

Feedback Loops

Evolving Organizational Structures
This is why the Point should remain broader than the name of any single technology.

39. Why "Policy-Driven Structural Hybrid Intelligence"?

Each term has a specific role.

Policy-Driven

Because runtime composition requires control.

Structural

Because the system is organized, not merely aggregated.

Hybrid

Because heterogeneous intelligence mechanisms coexist.

Intelligence

Because the object of study exceeds any single model or algorithm.
Together:
Policy-Driven Structural Hybrid Intelligence

captures the emerging architecture without claiming that its final form is already fixed.

40. Why “Toward”?

The term **Toward** is intentional.
The architecture described here is not yet a finished formal system.
Several questions remain open:
What is the minimal reference architecture?

What is the correct Policy representation?

How should local and global policies interact?

How should solver generation be triggered?

How should strategy spaces be bounded?

How should runtime stability be maintained?

How should structural provenance be represented?

How should successful compositions become reusable primitives?
Therefore this article defines a research direction, not a completed theory.

41. A Minimal Reference Architecture

A minimal implementation could contain:

```
Input
  |
  v
Structural Organizer
  |
  v
Node
  |
  +---- Algorithm
  |
  +---- ANN
  |
  +---- Two-Way CCC
  |
```

+---- LLM
|
v
Policy Router
|
v
Runtime
|
v
Measure
|
v
Feedback
This alone would be enough to test several core claims.

42. Minimal Policy Experiment

A first experiment might compare:
Static Solver
versus:
Rule-Based Policy
versus:
Learned Local Policy
over the same node.
Measures could include:
Accuracy

Cost

Latency

Solver Usage

Fallback Rate

Validation Rate

Policy Regret
This would provide a concrete entry into Point 12 research.

43. Minimal Intelligence-Generation Experiment

A second experiment could allow a node to generate new intelligence.
For example:
Local Data Threshold Reached
|
v
Train ANN
or:
Stable Positive / Negative Structure Detected
|
v
Construct Two-Way CCC
Then compare:
Before Generation

vs.

After Generation
This would test Learn-on-Demand Structural Hybrid Intelligence.

44. Minimal Composition Experiment

A third experiment could compare:
ANN Only
CCC Only
LLM Only
with:
ANN -> CCC
CCC -> LLM
ANN -> LLM -> Validator
This would make the Policy Surface directly measurable.

45. Structural Provenance

A hybrid system should preserve the path by which a result was produced.
For example:
Input X
|
v
Differential Node D4
|
v
Policy P7

|
v
ANN2
|
v
CCC Validator
|
v
Output Y
|
v
Measure M
This creates a runtime provenance chain.
Such traceability may become essential for debugging, validation, and scientific comparison.

46. Policy Provenance

The system should also preserve:
Why was ANN2 selected?

Why was CCC invoked?

Which constraint caused fallback?

Which measurement changed policy?
This means that Policy itself requires provenance.
Policy-driven systems should therefore record not only outcomes, but control decisions.

47. Local and Global Evolution

A hybrid ecology may evolve at two scales.

Local Evolution

Node N
->
new ANN
->
new CCC
->
new policy

Global Evolution

Network
  ->
new routing
  ->
new Brain Unit
  ->
new Calling Path
The interaction between these scales may become an important SI research area.

48. Structural Synthesis as an Optimization Problem

Once many structures exist, synthesis itself becomes an optimization problem.
The system may search over:
Component Choice

Component Order

Component Parameters

Routing

Validation

Runtime Budget
Thus:
Structural Components
        |
        v
Composition Space
        |
        v
Policy Optimization
        |
        v
Runtime Architecture
This directly connects Structural Synthesis with Enumerative Structure.

49. Policy-Driven Structural Hybrid Intelligence as a Meta-Architecture

The emerging system should not be interpreted as one universal implementation.
It is better understood as a **meta-architecture**.
It specifies computational roles:
Ground

Represent

Enumerate

Organize

Localize

Select

Compose

Execute

Measure

Adapt
Different applications may implement those roles differently.
This preserves generality without requiring one monolithic design.

50. Application Domains

Potential domains include:

- hybrid AI systems;
- autonomous systems;
- robotics;
- scientific discovery;
- personalized medicine;
- AI coding;

- industrial automation;
- decision support;
- knowledge systems;
- adaptive software;
- multi-agent systems.

The common requirement is:
multiple computational mechanisms must be organized under runtime constraints.

51. Boundary of the Claim

This article does **not** claim that:

- Point 12 has reached a final definition;
- Hybrid AI requires one specific architecture;
- policy optimization automatically improves every system;
- all intelligence should be decomposed into Brain Units;
- LLMs, ANN, CCC, search, and humans are interchangeable;
- every runtime node requires adaptive policy;
- Structural Hybrid Intelligence is already experimentally validated.

The narrower claim is:
The convergence of multiple Structural Intelligence mechanisms naturally creates computational junctions, strategy spaces, and policy surfaces, suggesting a broader synthesis layer centered on the runtime organization of heterogeneous intelligence.
This synthesis layer is provisionally described as **Policy-Driven Structural Hybrid Intelligence**.

52. The Candidate Point 12 Principle

A concise candidate principle is:

Point 12 studies what happens when heterogeneous forms of intelligence are allowed to work together under structural organization and policy control.

A second formulation is:

The next frontier may not be another isolated form of intelligence, but learning how existing forms of intelligence should be structurally organized, selectively invoked, and continuously optimized.

These formulations should remain hypotheses until further experimentation clarifies the architecture.

53. Conclusion

The Structural Intelligence Compass began by identifying important computational structures.

Those structures now increasingly point toward one another.

Grounded models create usable world structure.

Enumeration creates possibility spaces.

Differential Trees organize those spaces.

Calling Graphs expose computational reachability.

LLMs provide large-scale folded intelligence.

ANNs provide specialized local learning.

Two-Way CCC provides explicit structural recognition and triggering.

Policy determines how these capabilities interact.

Runtime determines what actually executes.

Measurement and feedback allow adaptation.

Together:

Ground
 |
 v
Represent
 |
 v
Enumerate
 |
 v
Organize
 |
 v
Localize
 |
 v

Expose Intelligence Space
  |
  v
Apply Policy
  |
  v
Execute
  |
  v
Measure
  |
  v
Adapt

This is the emerging shape of **Policy-Driven Structural Hybrid Intelligence**.

Closing Perspective

The first stage of Structural Intelligence asked:

What computational structures exist?

The next stage asks:

How should those structures work together?

And a future stage may ask:

How should successful runtime organizations evolve into new computational structures?

This produces the progression:

Structural Discovery
      ->
Structural Synthesis
      ->
Structural Evolution

Point 12 appears increasingly likely to sit at this transition.

Not as another isolated technology.

Not as a final universal model.

But as the place where previously separate computational structures begin to form an organized runtime whole.

That may be the deeper meaning of **Hybrid Structural Intelligence**:

Intelligence not as one winning mechanism, but as a policy-governed organization of complementary computational structures.